

# CF Recommender

## Codeforces Problem Recommendation System

### Technical Design & Implementation Report

---

|                      |                                 |
|----------------------|---------------------------------|
| <b>Version</b>       | 2.0.0                           |
| <b>Backend Stack</b> | Python 3.13 • FastAPI • Uvicorn |
| <b>Frontend</b>      | Vanilla HTML / CSS / JavaScript |
| <b>Data Source</b>   | Codeforces Public REST API      |
| <b>Date</b>          | June 2026                       |

---

This report documents the complete architecture, algorithmic design, and implementation of a personalised Codeforces practice-problem recommender.

The system combines content-based filtering, TF-IDF tag weighting, exponential decay modelling, and a UCB1 multi-armed bandit to generate high-quality, adaptive problem suggestions for competitive programmers.

# Contents

|       |                                                        |    |
|-------|--------------------------------------------------------|----|
| 1     | Introduction                                           | 2  |
| 2     | Project Structure                                      | 2  |
| 3     | Data Layer                                             | 3  |
| 3.1   | Data Models ( <code>models.py</code> )                 | 3  |
| 3.2   | Codeforces API Client ( <code>data_fetcher.py</code> ) | 3  |
| 3.2.1 | TTL Cache                                              | 3  |
| 3.2.2 | API Fetchers                                           | 4  |
| 3.2.3 | Index Structures                                       | 4  |
| 3.2.4 | Candidate Generation                                   | 5  |
| 3.2.5 | Rating Helper                                          | 5  |
| 4     | Recommendation Engine                                  | 5  |
| 4.1   | User Profile                                           | 5  |
| 4.1.1 | Exponential Decay                                      | 5  |
| 4.1.2 | Wrong-Submission Penalty                               | 6  |
| 4.1.3 | Tag Accuracy                                           | 6  |
| 4.2   | Problem Scoring                                        | 6  |
| 4.2.1 | Difficulty Match                                       | 6  |
| 4.2.2 | Weak-Tag Boost (IDF-Weighted)                          | 6  |
| 4.2.3 | Recency Bonus                                          | 7  |
| 4.3   | Top- $k$ Selection (Min-Heap)                          | 7  |
| 5     | UCB1 Multi-Armed Bandit                                | 7  |
| 5.1   | Motivation                                             | 8  |
| 5.2   | Algorithm                                              | 8  |
| 5.3   | Exploit/Explore Split                                  | 8  |
| 5.4   | Persistence                                            | 8  |
| 6     | REST API ( <code>api.py</code> )                       | 9  |
| 6.1   | Framework                                              | 9  |
| 6.2   | Server Startup                                         | 9  |
| 6.3   | Endpoints                                              | 9  |
| 6.4   | Response Schemas                                       | 10 |
| 6.5   | CORS                                                   | 10 |
| 7     | Frontend ( <code>cf_recommender_frontend.html</code> ) | 10 |
| 7.1   | Technology                                             | 10 |
| 7.2   | Design System                                          | 10 |
| 7.3   | User Interaction Flow                                  | 10 |
| 7.4   | Rating Colour Coding                                   | 11 |
| 7.5   | Animated Background                                    | 11 |
| 8     | End-to-End Recommendation Pipeline                     | 11 |
| 9     | Complexity Analysis                                    | 12 |
| 10    | Limitations and Future Work                            | 13 |
| 10.1  | Current Limitations                                    | 13 |
| 10.2  | Potential Improvements                                 | 14 |
| 11    | Summary                                                | 14 |

---

## 1 Introduction

---

Codeforces is one of the largest competitive-programming platforms in the world, hosting tens of thousands of rated problems across thousands of contests. A typical user has solved anywhere from a few dozen to several thousand of those problems and faces the question: *which problem should I practise next?*

Answering that question well requires reasoning about at least three things simultaneously:

1. **Difficulty calibration** — Problems should be hard enough to stretch the user but not so hard as to be demoralising or unhelpful.
2. **Weakness targeting** — The user’s performance is uneven across algorithmic topics (e.g. they may be strong at greedy but weak at segment trees). Good recommendations surface problems in their weakest areas.
3. **Exploration vs. exploitation** — A pure greedy approach exploits known weaknesses but never ventures into unfamiliar territory. Genuine growth also requires occasionally exploring novel problem types.

The **CF Recommender** addresses all three concerns within a clean, two-component architecture: a Python backend (FastAPI) that encapsulates all algorithmic logic, and a single-page HTML/JS frontend that consumes the backend’s REST endpoints.

The recommender uses a user’s submission history and problem metadata to build a personalized practice profile. By analysing topic-wise performance instead of relying only on ratings, it aims to suggest problems that are both relevant and beneficial for skill improvement.

---

## 2 Project Structure

---

```
cf_recommender/  
  backend/  
    models.py           # Dataclasses & shared constants  
    data_fetcher.py     # Codeforces API client, indexing, candidate generation  
    recommender.py      # Scoring engine & UCB1 bandit  
    main.py             # Command-line entry point  
    api.py              # FastAPI REST server  
    bandit_states/      # Per-user UCB1 state (JSON files)  
  frontend/  
    cf_recommender_frontend.html  # Single-page UI
```

The separation is deliberate: `models.py` defines data contracts that every other module imports, `data_fetcher.py` owns all I/O and index construction, `recommender.py` owns all machine-learning/algorithmic logic, and `api.py` is pure glue between them.

### 3 Data Layer

#### 3.1 Data Models (`models.py`)

Five dataclasses form the backbone of the system:

| Class                      | Purpose                                                                                                                                                                                                                                                   |
|----------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>Problem</code>       | A single Codeforces problem: <code>contest_id</code> , <code>index</code> (e.g. “B”), <code>rating</code> , <code>tags</code> , <code>name</code> . The computed property <code>pid</code> produces the canonical <i>contest-index</i> key (e.g. 1536-F). |
| <code>Submission</code>    | One submission attempt: a <code>Problem</code> , a <code>verdict</code> string (“OK”, “WRONG_ANSWER”, etc.), and a Unix timestamp.                                                                                                                        |
| <code>RatingEvent</code>   | A single contest result: <code>contest_id</code> , <code>new_rating</code> , Unix timestamp.                                                                                                                                                              |
| <code>UserProfile</code>   | The processed user state used by the scoring engine (see Section 4.1).                                                                                                                                                                                    |
| <code>ScoredProblem</code> | A candidate problem annotated with its final score and recommendation mode (“exploit” or “explore”).                                                                                                                                                      |

Table 1: Core data model classes

Three module-level constants govern algorithm behaviour:

| Constant                            | Value | Meaning                                               |
|-------------------------------------|-------|-------------------------------------------------------|
| <code>TARGET_RATING_OFFSET</code>   | 100   | Recommend problems 100 points below the user’s rating |
| <code>RATING_SCALE</code>           | 400   | Normalisation window for difficulty-match scoring     |
| <code>REGENCY_HALF_LIFE_DAYS</code> | 90    | Exponential decay half-life for solved problems       |

Table 2: Shared algorithmic constants

#### 3.2 Codeforces API Client (`data_fetcher.py`)

##### 3.2.1 TTL Cache

All HTTP requests are routed through a module-level dictionary cache keyed by URL. A cached entry is considered fresh for `CACHE_TTL = 3600` seconds (one hour). This prevents repeated Codeforces API calls within a single server session and avoids hitting Codeforces’s rate limits.

```

1 _cache: dict[str, tuple[dict, float]] = {}
2
3 def cached_get(url: str) -> dict:
4     now = time.time()
5     if url in _cache:

```

```

6         data, ts = _cache[url]
7         if now - ts < CACHE_TTL:
8             return data
9         response = requests.get(url, timeout=10)
10        response.raise_for_status()
11        data = response.json()
12        _cache[url] = (data, now)
13    return data

```

Listing 1: TTL cache implementation

### 3.2.2 API Fetchers

Three public functions wrap Codeforces REST endpoints:

- **get\_user\_submissions(handle)** — Calls `/user.status` and returns a list of `Submission` objects. Skips problems without a `contestId` (these are gym or practice-only problems).
- **get\_all\_problems()** — Calls `/problemset.problems` and returns the full rated problemset (problems without a rating or contest ID are excluded).
- **get\_user\_rating(handle)** — Calls `/user.rating` and returns a list of `RatingEvent` objects representing the user’s full contest history.

### 3.2.3 Index Structures

After loading the full problemset the system builds three in-memory indexes:

**Tag inverted index.** Maps each topic tag to the list of problem IDs that carry it. Candidate generation (Section 3.2.4) uses this to avoid an  $\mathcal{O}(n \times m)$  brute-force scan.

```

1 def build_tag_index(problemset: dict[str, Problem]) -> dict[str,
2     list[str]]:
3     index: dict[str, list[str]] = defaultdict(list)
4     for pid, prob in problemset.items():
5         for tag in prob.tags:
6             index[tag].append(pid)
7     return dict(index)

```

Listing 2: Inverted tag index construction

**Rating index.** Two parallel structures support  $\mathcal{O}(\log n)$  rating-range queries using Python’s `bisect` module: a sorted list of unique ratings and a mapping from each rating to the corresponding problem IDs.

**TF-IDF tag weights.** An Inverse Document Frequency (IDF) score is computed for every tag:

$$\text{idf}(t) = \ln\left(\frac{N}{\text{df}(t)}\right)$$

where  $N$  is the total number of problems and  $\text{df}(t)$  is the number of problems tagged with  $t$ .

Common tags such as *implementation* and *greedy* appear in thousands of problems and receive a low IDF weight; rare, specific tags like *centroid decomposition* appear in far fewer problems and receive a high weight. This prevents generic tags from dominating the scoring.

### 3.2.4 Candidate Generation

Rather than scoring the entire problemset for every request, the system first generates a focused candidate pool:

1. Retrieve the user's  $\leq 50$  most recently solved problems (the “seed”).
2. For each seed problem, look up all problems sharing at least one tag via the inverted index.
3. Take the union of all results and remove problems already solved by the user.

The resulting pool is typically in the low thousands — large enough for good coverage but small enough for fast scoring. Complexity is  $\mathcal{O}(\text{recent} \times \overline{\text{tags}} \times \overline{\text{bucket}})$  rather than  $\mathcal{O}(\text{recent} \times |\text{problemset}|)$ .

### 3.2.5 Rating Helper

`compute_recent_cf_rating` averages contest results from the last 30 days. If the user has not participated in any recent contest, it falls back to their most recent historical rating. This produces a “live” rating that better reflects the user's current ability than the official displayed rating, which can lag behind.

## 4 Recommendation Engine

### 4.1 User Profile

The `build_user_profile` function constructs a `UserProfile` from the user's raw submission history. This is the most analytically rich step in the pipeline.

#### 4.1.1 Exponential Decay

Competitive programming skill is not static. A problem solved two years ago on a topic the user has not revisited counts for less than a problem solved last month. The system models this with exponential decay:

$$w(t) = e^{-\frac{\ln 2 \cdot \Delta t}{H}}$$

where  $\Delta t$  is the age of the submission in days and  $H = 90$  days is the half-life.

Under this model, a solve from today has weight 1.0, a solve from 90 days ago has weight 0.5, and a solve from 180 days ago has weight 0.25.

#### 4.1.2 Wrong-Submission Penalty

Solving a problem after 5 wrong attempts indicates lower mastery than solving it on the first try. Each wrong submission before an accepted solution applies a fractional penalty:

$$\text{penalty}(w) = \frac{1}{1 + 0.3w}$$

where  $w$  is the number of wrong submissions before the accepted one.

The final per-solve weight is  $w(t) \times \text{penalty}(w)$ .

#### 4.1.3 Tag Accuracy

For each tag, the profile stores:

- **tag\_accuracy** — Weighted fraction of problems solved, in  $[0, 1]$ , where the weight is the decay-and-penalty value above. Tags where every attempt was wrong have accuracy 0 (tracked separately as “unsolved” tags, not dropped).
- **tag\_attempts** — Raw count of unique problems the user has touched that carry this tag (used by UCB1).
- **unsolved\_tags** — Tags present in the global problemset that the user has never attempted at all, sorted alphabetically.

The **target rating** is defined as `current_rating` – 100, i.e. problems slightly below the user’s rating. This is intentional: since the recommended problems are already drawn from the user’s weaker tags, a slightly lower rating is used to balance the additional difficulty introduced by those topics, ensuring the problems remain challenging yet approachable.

## 4.2 Problem Scoring

Each candidate problem first receives a composite score:

$$\text{score}(p, u) = S_{\text{difficulty}}(p, u) + S_{\text{weakness}}(p, u)$$

#### 4.2.1 Difficulty Match

$$S_{\text{difficulty}}(p, u) = \max\left(0, 1 - \frac{|r_p - r^*|}{\text{RATING\_SCALE}}\right)$$

where  $r_p$  is the problem’s rating and  $r^* = \text{current\_rating} - 100$  is the target rating.

Problems within 400 rating points of the target score positive; those farther away score 0. The candidate set is also pre-filtered to  $|r_p - r^*| \leq 400$  before scoring begins, so extreme outliers are never even considered.

#### 4.2.2 Weak-Tag Boost (IDF-Weighted)

$$S_{\text{weakness}}(p, u) = \sum_{t \in \text{tags}(p)} \left(1 - \text{accuracy}_u(t)\right) \cdot \frac{\text{idf}(t)}{\text{idf}_{\max}}$$

The key properties of this formula are:

- Tags the user has never attempted (**accuracy** defaults to 0.0) receive the maximum boost of 1.
- Tags the user has mastered (**accuracy**  $\approx 1.0$ ) contribute nearly nothing.
- IDF normalisation ensures that rare, specific tags (e.g. “palindrome tree”) weight more than generic ones (e.g. “implementation”).

#### 4.2.3 Recency Bonus

After both components are computed, a final recency bonus is added to favour newer contest problems:

$$\text{final\_score}(p, u) = \frac{\text{score}(p, u)}{\text{score}_{\max}} + 0.2 \cdot \frac{\text{contest\_id}(p)}{\text{contest\_id}_{\max}}$$

The base score is normalised to  $[0, 1]$  before the bonus is added. Contest IDs on Codeforces are monotonically increasing, so higher IDs correspond to more recent contests. The bonus is capped at 0.2 to prevent it from overwhelming the content-based signal. So we can conclude the final score of a problem lies in the range  $[0, 1.2]$ .

### 4.3 Top- $k$ Selection (Min-Heap)

Instead of sorting all  $n$  scored candidates to find the top  $k$ , the system maintains a min-heap of size  $k$ :

```

1 heap: list[tuple[float, str]] = []
2
3 for pid, prob in unsolved.items():
4     s = final_score(pid, prob)
5     if len(heap) < k:
6         heapq.heappush(heap, (s, pid))
7     elif s > heap[0][0]:
8         heapq.heapreplace(heap, (s, pid))
9
10 top = sorted(heap, reverse=True)
```

Listing 3:  $O(n \log k)$  top- $k$  selection with a min-heap

This is  $\mathcal{O}(n \log k)$  rather than  $\mathcal{O}(n \log n)$ . For typical values ( $n \approx 3,000$ ,  $k = 10$ ) the constant-factor improvement is roughly  $3\times$ .

## 5 UCB1 Multi-Armed Bandit

## 5.1 Motivation

The scoring engine (Section 4) is a static function of a problem’s tags and rating. It does not learn from the user’s behaviour over time. A user might consistently solve “implementation” problems recommended to them while always skipping “constructive algorithms” problems — yet without feedback, the system cannot adjust.

The UCB1 (Upper Confidence Bound 1) algorithm solves this by modelling the recommendation task as a multi-armed bandit: each problem is an “arm”, pulling it means recommending it, and the reward is whether the user solved it.

## 5.2 Algorithm

$$\text{UCB}_i(t) = \bar{x}_i + \sqrt{\frac{2 \ln t}{n_i}}$$

where  $\bar{x}_i$  is the running-average reward for problem  $i$ ,  $t$  is the total number of recommendations made so far, and  $n_i$  is the number of times problem  $i$  has been recommended.

The exploration bonus  $\sqrt{2 \ln t / n_i}$  is large when a problem has been recommended few times (high uncertainty) and shrinks as it is shown more. Problems never shown yet have  $n_i = 0$ , giving them infinite UCB — they are always prioritised on first encounter.

**Why UCB1 over  $\epsilon$ -greedy.** The earlier version of the system used  $\epsilon$ -greedy exploration: with probability  $\epsilon$  pick a random problem, otherwise pick the best-scored one. UCB1 is strictly better here because its exploration is principled — it explores in proportion to uncertainty — whereas  $\epsilon$ -greedy explores uniformly at random, wasting recommendations on problems that have already been well-characterised.

## 5.3 Exploit/Explore Split

The final recommendation list of  $k = 10$  problems is assembled as follows:

- **9 exploit** (90%): Problems from the top- $k$  scored list, selected via UCB1. This blends scoring-based quality with UCB-based exploration within the ranked pool.
- **1 explore** (10%): A uniformly random problem from the right difficulty range that was *not* in the scored pool at all. This prevents the system from getting stuck in a “tag bubble” of problems similar to what the user has already practised.

Explore recommendations are labelled with `mode = "explore"` and displayed with a distinct visual badge in the frontend.

## 5.4 Persistence

Each user’s bandit state (`counts`, `values`, `t`) is serialised to a JSON file under `bandit_states/<handle>.json` after every request. On the next request the state is deserialised and updated. This means the bandit’s learned preferences survive server restarts and accumulate over the user’s lifetime.

```

1 def load_bandit(handle: str) -> UCB1Bandit:
2     path = os.path.join(BANDIT_DIR, f"{handle}.json")
3     bandit = UCB1Bandit()
4     if os.path.exists(path):
5         with open(path) as f:
6             data = json.load(f)
7             bandit.counts = data["counts"]
8             bandit.values = data["values"]
9             bandit.t = data["t"]
10    return bandit

```

Listing 4: Bandit state persistence (load)

## 6 REST API (`api.py`)

### 6.1 Framework

The backend is a **FastAPI** application. FastAPI was chosen for its automatic OpenAPI documentation, Pydantic-based request/response validation, and modern `async` lifespan management.

### 6.2 Server Startup

The `lifespan` context manager (FastAPI's modern replacement for `@app.on_event("startup")`) fetches the full Codeforces problemset once when the server starts, builds all three indexes (tag, rating, IDF), and stores them in `app.state`. All subsequent requests reuse these structures without making another API call.

### 6.3 Endpoints

| Route                            | Method | Description                                                                                                                                                                                                                                                |
|----------------------------------|--------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>/recommend/{handle}</code> | GET    | Returns a list of up to $k$ recommended problems. Runs the full pipeline: fetch submissions $\rightarrow$ build profile $\rightarrow$ generate candidates $\rightarrow$ score $\rightarrow$ UCB1 selection. Bandit state is loaded before and saved after. |
| <code>/feedback</code>           | POST   | Accepts <code>{handle, pid, solved}</code> and updates the user's bandit state with reward 1.0 (solved) or 0.0 (skipped/failed).                                                                                                                           |
| <code>/profile/{handle}</code>   | GET    | Returns the user's current rating, target rating, top-10 weakest tags with accuracy and attempt counts, and the full list of never-attempted tags.                                                                                                         |

Table 3: REST API endpoints

## 6.4 Response Schemas

All responses are validated via Pydantic models. The `RecommendedProblem` schema adds a `url` field built by `pid_to_url()` which converts a PID such as 1536-F into the canonical Codeforces URL `https://codeforces.com/problemset/problem/1536/F`.

## 6.5 CORS

Cross-Origin Resource Sharing is currently set to allow all origins (`allow_origins=["*"]`), which is appropriate for local development and demo deployments. A production deployment should restrict this to the frontend's actual domain.

# 7 Frontend (`cf_recommender_frontend.html`)

## 7.1 Technology

The frontend is a single, self-contained HTML file with no build step and no external JavaScript dependencies. It relies on:

- **Vanilla JS (ES2020)** for all application logic.
- **CSS custom properties** for a consistent dark-mode design system.
- **Google Fonts** (Space Mono, Syne) for typography.
- The browser's **Fetch API** for all backend communication.

## 7.2 Design System

The UI uses a dark colour palette with a purple/indigo accent (`#7c6af7`) and a CSS variable-based design system. Key variables include:

```

1 --bg:      #0a0a0f      /* page background */
2 --surface: #111118      /* card background  */
3 --accent:  #7c6af7      /* primary accent   */
4 --green:   #34d399      /* success / explore */
5 --amber:   #fbbf24      /* medium difficulty */
6 --red:     #f87171      /* wrong / error     */

```

Listing 5: CSS design tokens

## 7.3 User Interaction Flow

1. The user types their Codeforces handle into the search bar and clicks “Analyse” (or presses Enter).
2. Two parallel `fetch` calls are fired: one to `/recommend/{handle}` and one to `/profile/{handle}`.
3. While loading, skeleton placeholder cards animate to give immediate visual feedback.

4. On success, problem cards animate into view one by one (`fadeUp` animation with staggered delays).
5. Each problem card displays: rank, problem name, problem ID, rating badge (colour-coded by Codeforces tier), topic tags, and an exploit/explore mode badge.
6. Two feedback buttons (“✓ solved” and “× skip”) call `POST /feedback` and toggle their active state in the UI.
7. The sidebar shows the user’s profile card (handle, current rating, target rating, unsolved tag count, list of never-attempted tags) and a weakest-tags bar chart with a tooltip explaining the decay-based “tag readiness” metric.

## 7.4 Rating Colour Coding

Problem ratings are colour-coded to match Codeforces’ own rank colour scheme:

| Rating Range | CF Rank          | Colour Hex | Visual |
|--------------|------------------|------------|--------|
| < 1200       | Newbie           | #888888    | Grey   |
| 1200–1399    | Pupil            | #34d399    | Green  |
| 1400–1599    | Specialist       | #60a5fa    | Blue   |
| 1600–1899    | Expert           | #a78bfa    | Violet |
| 1900–2099    | Candidate Master | #fbbf24    | Amber  |
| 2100–2399    | Master           | #fb923c    | Orange |
| ≥ 2400       | Grandmaster      | #f87171    | Red    |

Table 4: Rating tier colour coding

## 7.5 Animated Background

The landing page features a subtle animated hero section with floating problem cards that drift upwards, loop, and fade out. These use dummy problem data (hardcoded `fakeProblems` array) and are spawned at 1.8-second intervals via `setInterval`. They serve purely as visual decoration and convey the system’s purpose without any backend calls.

# 8 End-to-End Recommendation Pipeline

The complete pipeline, from an incoming API request to a final response, is:

**Step 1. Fetch submissions** — Call `/user.status` (cached). Extract solved set and per-problem metadata.

**Step 2. Fetch rating history** — Call `/user.rating` (cached). Compute recent-average rating.

- 
- Step 3. Build user profile** — For each submitted problem, accumulate decay-weighted, penalty-adjusted solved counts per tag. Compute tag accuracy and identify unsolved tags.
- Step 4. Generate candidate pool** — Use the last 50 solved problems as seeds. Expand via the inverted tag index. Remove already-solved problems. This yields  $\sim 2,000$ – $5,000$  candidate PIDs.
- Step 5. Pre-filter by rating** — Restrict candidates to problems within  $\pm 400$  of the target rating.
- Step 6. Score candidates** — Compute  $S_{\text{difficulty}} + S_{\text{weakness}}$  for each candidate. Normalise base scores. Add recency bonus.
- Step 7. Select top- $k$**  — Use a min-heap of size  $k$  for  $\mathcal{O}(n \log k)$  selection.
- Step 8. UCB1 selection** — Load the user’s bandit state. Select 9 exploit problems via UCB1 from the top- $k$  pool. Sample 1 explore problem randomly from outside the pool.
- Step 9. Save bandit state** — Serialise updated UCB1 state to disk.
- Step 10. Return response** — Annotate each selected problem with its URL and return the Pydantic-validated list.

## 9 Complexity Analysis

---

Let  $N$  be the total number of problems in the Codeforces problemset (currently  $\approx 10,000$ ),  $M$  be the number of user submissions,  $R$  be the number of recent solved seeds (max 50),  $\bar{T}$  be the average number of tags per problem ( $\approx 3$ ), and  $\bar{B}$  be the average tag bucket size.

| Operation            | Description                                       | Complexity                              |
|----------------------|---------------------------------------------------|-----------------------------------------|
| Profile build        | Iterate all submissions, accumulate per-tag stats | $\mathcal{O}(M\bar{T})$                 |
| Tag index build      | Iterate problemset, insert into hash map          | $\mathcal{O}(N\bar{T})$                 |
| Rating index build   | Sort unique ratings                               | $\mathcal{O}(N \log N)$                 |
| IDF computation      | One pass over problemset                          | $\mathcal{O}(N\bar{T})$                 |
| Candidate generation | Union over tag buckets of seeds                   | $\mathcal{O}(R\bar{T}\bar{B})$          |
| Rating range query   | Binary search on sorted ratings                   | $\mathcal{O}(\log N)$                   |
| Candidate scoring    | Score each candidate                              | $\mathcal{O}(C\bar{T})$ where $C \ll N$ |
| Top- $k$ selection   | Min-heap maintenance                              | $\mathcal{O}(C \log k)$                 |
| UCB1 selection       | Scan top- $k$ list                                | $\mathcal{O}(k)$                        |

Table 5: Computational complexity of key pipeline stages

All startup indexes are built once and reused across requests. The per-request cost is dominated by **candidate scoring**, which is  $\mathcal{O}(C\bar{T})$  over a pool of  $C \approx 2,000\text{--}5,000$  candidates with  $\bar{T} \approx 3$  tags each — comfortably under one millisecond of CPU time.

## 10 Limitations and Future Work

### 10.1 Current Limitations

- **No collaborative filtering.** The system is purely content-based and does not exploit similarities between users. A user whose tag history is sparse (e.g. a new account) receives weaker recommendations.
- **In-process cache.** The TTL cache is held in module-level memory. Under a multi-process deployment (multiple Uvicorn workers) each process maintains its own cache, causing redundant API calls. A shared Redis cache would fix this.
- **Bandit state race condition.** Under concurrent requests for the same handle, two processes could load the same bandit state, update it independently, and the later save would overwrite the earlier one. This is unlikely in practice (each user session is sequential) but a proper database or file lock would remove the risk.
- **Tag labelling quality.** Codeforces tags are user-contributed and not always consistent. A problem labelled “greedy” by one contest may use nearly identical reasoning to a problem labelled “constructive algorithms” by another, causing the tag-accuracy model to treat them as independent skills.
- **Static explore split.** The 90/10 exploit/explore split is fixed. An adaptive schedule (larger explore early in a user’s history, smaller later) could improve convergence.

## 10.2 Potential Improvements

- **Collaborative filtering layer.** Use matrix factorisation or a KNN approach over user-tag accuracy matrices to incorporate community-level patterns.
- **Async API calls.** Replace synchronous `requests` with `httpx` and `async/await` to allow the FastAPI server to handle more concurrent users without blocking.
- **Problem difficulty model.** Codeforces problem ratings are crowd-sourced estimates and can be noisy. A learned difficulty model trained on acceptance rates could provide more stable difficulty signals.
- **Contest-mode simulation.** Recommend a set of problems that mimics a virtual contest (problems A–F with increasing difficulty) so the user can practice under contest conditions.
- **Authentication.** Integrating Codeforces OAuth or a lightweight account system would allow cross-device bandit state without sharing a handle.

## 11 Summary

The CF Recommender is a well-engineered, algorithmically rich system that delivers personalised Codeforces problem suggestions through a clean, two-tier architecture. Its key technical contributions are:

1. **Decay-weighted tag accuracy modelling** that captures the user’s *current* skill level by down-weighting old solves and penalising problems solved after many wrong attempts.
2. **IDF-weighted weak-tag scoring** that ensures rare, specific topics (which often represent genuine knowledge gaps) contribute more to the recommendation signal than ubiquitous tags.
3. **Efficient  $O(n \log k)$  top- $k$  selection** via a min-heap, avoiding a full sort of the candidate set.
4. **UCB1 multi-armed bandit** for principled exploration/exploitation that adapts to each user’s engagement patterns over time.
5. **Persistent bandit state** that accumulates across sessions, making the system progressively smarter for returning users.
6. **Inverted tag and binary-search rating indexes** that keep per-request latency low even against a problemset of tens of thousands of entries.

The result is a system that is simultaneously *theoretically grounded* (in information retrieval and bandit theory), *computationally efficient*, and *practically useful* for any competitive programmer looking to practise more deliberately.